



M. Kumarasamy
College of Engineering

Autonomous | Affiliated to Anna University Chennai

Approved by AICTE, Accredited by NBA and NAAC
Thalavapalayam, Karur - 639 113, Tamilnadu, India.



A Project Report

on

Payroll Processing System

Submitted in partial fulfillment of requirements for the award of the course

of

CGB1121-PYTHON PROGRAMMING

Under the guidance of

Mrs. I . KARTHIKA M.E
Assistant Professor / CSE

Submitted By

KANISHKA A K (927625BCS064)

PATHMASRI V M (927625BCS116)

DEPARTMENT OF FRESHMEN ENGINEERING

M.KUMARASAMY COLLEGE OF ENGINEERING
(Autonomous)

KARUR – 639 113

MAY 2026

M. KUMARASAMY COLLEGE OF ENGINEERING
(Autonomous Institution affiliated to Anna University, Chennai)

KARUR – 639 113

BONAFIDE CERTIFICATE

This is to certify that this project report on “**PAYROLL PROCESSING SYSTEM**” is the bonafide work of **KANISHKA A K(927625BCS064)**, **PATHMASRI V M (927625BCS116)** who carried out the project work during the academic year 2025 - 2026 under my supervision.

Signature

Mrs. I. KARTHIKA, M.E.,

Supervisor,

Department of Computer Science &
Engineering,

M. Kumarasamy College of
Engineering,

Thalavapalayam, Karur – 639 113.

Signature

Dr. K. CHITIRAKALA, M.Sc.,

M.Phil., Ph.D.,

Head Freshmen Engineering,

Department of Freshmen Engineering,

M. Kumarasamy College of
Engineering,

Thalavapalayam, Karur – 639 113.

Submitted for the End Semester Review held on_____.

INTERNAL EXAMINER

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education

MISSION OF THE INSTITUTION

- Produce smart technocrats with empirical knowledge who can surmount the global challenges
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations

VISION OF THE DEPARTMENT

To achieve education and research excellence in computer Science and Engineering

MISSION OF THE DEPARTMENT

- To excel in academic through effective teaching learning techniques.
- To promote research in the area of computer science and engineering with the focus on innovation.
- To transform students into technically competent professionals with societal and ethical responsibilities.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- **PEO1:** Graduates will have successful Career in software industries and R&D divisions through continuous learning
- **PEO2:** Graduates will provide effective solutions for real world problems in the key domain of computer science and engineering and engage in lifelong learning.
- **PEO3:** Graduates will excel in their profession by being ethically and socially responsible

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

1. **Engineering knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals, and an engineering specialization to develop solutions to complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
3. **Design/development of solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
4. **Conduct investigations of complex problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.
6. **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.
8. **Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

10. **Project management and finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11. **Life-long learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

- **PSO1: Professional Skills:** Ability to apply the knowledge of computing techniques to design and develop computerized solutions for the problems.
- **PSO2: Successful career:** Ability to utilize the computing skills and ethical values in creating a successful career.

ABSTRACT

The Payroll Processing System is a Python-based application developed to automate the calculation of employee salaries in an accurate and efficient manner. Manual salary computation can lead to errors and consume significant time, so this system provides a structured and reliable solution using Python programming concepts. The application uses separate functions to calculate basic pay, allowances, deductions, and net salary. By dividing the program into modular components, the system ensures better organization, easy maintenance, and improved accuracy. Allowances and deductions are calculated independently before determining the final net pay. The system takes employee details as input, processes all salary components step by step, and generates the final salary output. This project demonstrates the practical use of functions, arithmetic operations, and logical structures in Python to build a simple and effective payroll management system.

ABSTRACT WITH POs AND PSOs MAPPING

ABSTRACT	COs MAPPED	POs MAPPED	PSOs MAPPED
The Payroll Processing System is a Python-based program designed to automate employee salary calculation. It uses separate functions to compute basic pay, allowances, deductions, and net salary, ensuring accuracy and clarity. The system takes employee details as input and generates the final salary after processing all components. This project demonstrates the use of modular programming in Python to create a simple, reliable, and maintainable payroll management solution.	CO1	PO1(3)	PSO1(3)
	CO2	PO2(3)	PSO2(3)
	CO3	PO3(3)	
	CO4	PO4(2)	
		PO5(3)	
		PO6(1)	
		PO7(1)	
		PO8(2)	
		PO9(2)	
		PO10(2)	
		PO11(2)	

Note: 1- Low, 2-Medium, 3- High

SUPERVISOR

HEAD OF THE DEPARTMENT

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	vi
1	INTRODUCTION	1
	1.1 PROBLEM STATEMENT	1
	1.2 INTRODUCTION	2
	1.3 PYTHON PROGRAMMING CONCEPTS	3
	1.4 OBJECTIVE	4
2	PROJECT METHODOLOGY	5
	2.1 PROPOSED WORK	5
	2.2 ARCHITECTURE	6
3	SOFTWARE REQUIREMENTS	7
4	MODULE DESCRIPTION	8
	4.1 EMPLOYEE MANAGEMENT MODULE	8
	4.2 SALARY AND PAYROLL MODULE	8
	4.3 ALLOWANCES AND DEDUCTION MODULE	8
	4.4 DATABASE MANAGEMENT MODULE	8
	4.5 REPORTS AND SECURITY MODULE	8
5	IMPLEMENTATION	9
	5.1 CODING	9
6	SNAPSHOTS	18
	CONCLUSION	20
	REFERENCES	21

CHAPTER 1

INTRODUCTION

1.1PROBLEM STATEMENT

Here is a problem statement for Payroll Processing System in python programming:

Problem Statement

The management of employee payroll in organizations is often performed manually, which can lead to errors, delays, and difficulties in maintaining employee salary records. Existing methods may not ensure accurate salary calculations and proper management of employee details. Therefore, a system is needed to automate payroll processing and efficiently handle employee salary information. The system should provide an easy interface to manage employee data, calculate salaries, and generate payroll details accurately.

Requirements

User Input:

The system accepts user input such as employee details including employee ID, employee name, basic salary, attendance, overtime hours, allowances, and deductions. These inputs help the system identify employee information and perform payroll calculations effectively.

Processing:

The system processes the entered employee data using Python programming logic. It calculates the total salary by considering basic salary, overtime pay, bonuses, allowances, and deductions. The system also stores and manages employee payroll records for future reference.

Output:

Based on the given inputs, the system displays the calculated payroll information clearly, including employee details, gross salary, deductions, and net salary. It also generates payroll reports or salary slips in a structured format for easy understanding and record maintenance.

1.2 INTRODUCTION

The Payroll Processing System is a Python-based application developed to automate and manage employee salary operations efficiently. In many organizations, payroll calculation is a time-consuming and error-prone task when done manually. This system helps in calculating employee salaries by considering basic pay, allowances, bonuses, overtime, and deductions such as tax and provident fund. The project is developed using core Python concepts like functions, loops, conditional statements, and file handling to store and process employee records. It ensures accuracy, reduces paperwork, saves time, and improves overall payroll management. This system is especially useful for small and medium-sized organizations to maintain structured and reliable salary records.

1.3 PYTHON PROGRAMMING CONCEPTS

1. Variables & Data Types

- ▶ Store employee details and salary values

2. Conditional Statements (if-else)

- ▶ Calculate tax, deductions, and salary conditions

3. Loops (for / while)

- ▶ Process multiple employee records

4. Functions

- ▶ Perform modular salary calculations

5. File Handling

- ▶ Store and retrieve payroll data (CSV/TXT)

6. Data Structures (List / Dictionary)

- ▶ Manage employee records efficiently

7. Object-Oriented Programming

- ▶ Create Employee or Payroll classes

1.4 OBJECTIVE

The main objective of this project is to develop a Payroll Processing System using Python that automates the calculation of employee salaries in an accurate and efficient manner. The system aims to reduce manual errors, save time, and ensure proper management of employee payroll data. It is designed to handle employee details, calculate salary components such as basic pay, allowances, and deductions, and generate the final net salary. The project also focuses on providing a user-friendly interface and maintaining data securely using a database.

CHAPTER 2

PROJECT METHODOLOGY

2.1 PROPOSED WORK

The proposed system focuses on developing a Payroll Processing System using Python with a modular and efficient approach. The system is designed to manage employee details, calculate salaries, and generate payroll reports accurately. The development process includes the following steps:

► Requirement Analysis

Identify the main functionalities such as adding employee details, calculating salary, handling allowances and deductions, and generating payslips. The system should also ensure accuracy and reduce manual errors.

► System Design

Design a simple and modular architecture consisting of components like employee management module, salary calculation module, database module, and report generation module for better organization.

► Implementation

Develop the system using Python programming language with GUI support using Tkinter and database connectivity using MySQL to store and manage employee data efficiently.

► Testing and Validation

Test the system with different employee data inputs to ensure correct salary calculation and proper functioning of add, update, and delete operations. Validate accuracy and fix errors.

► Deployment

Make the application user-friendly and easy to run on standard systems using VS Code or any Python environment so that users can operate it without difficulty.

2.2 ARCHITECTURE

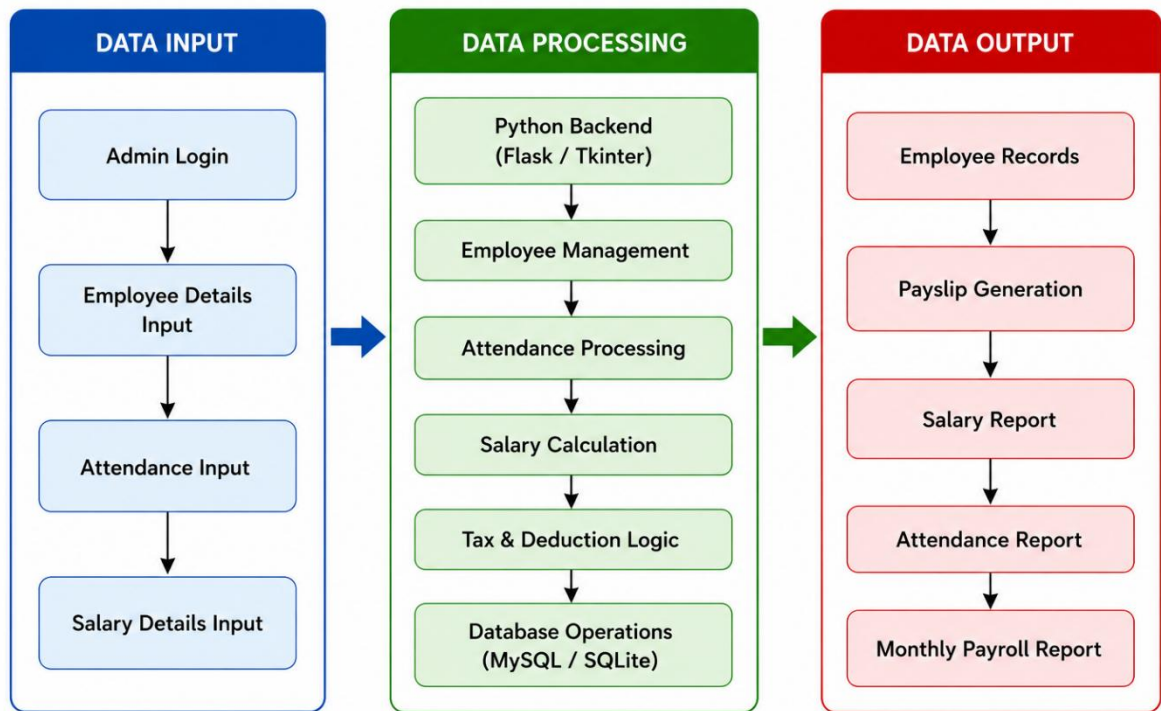


Fig 2.1 Architecture Diagram of Payroll Processing System

CHAPTER 3

SOFTWARE REQUIREMENTS

The project requires the following software components:

- **Python 3.8 :** Used as the main programming language for developing the payroll processing system.
- **Tkinter Library:** Used to design the graphical user interface (GUI) such as login page, forms, and buttons.
- **MySQL Database:** Used to store employee details, salary data, and login credentials securely.
- **mysql-connector-python:** Used to connect Python with MySQL database and execute SQL queries.
- **Python IDE:** Used for writing and executing the program (VS Code / PyCharm / IDLE).
- **Operating System:** Supports Windows, Linux, or macOS platforms.
- **Libraries:** tkinter, tkinter.ttk, tkinter.messagebox, mysql.connector

CHAPTER 4

MODULE DESCRIPTION

4.1 Employee Management Module

This module is used to manage employee details such as name, basic salary, and other relevant information. It allows adding, updating, and deleting employee records in the system.

4.2 Salary & Payroll Module

This module handles the calculation of employee salary by combining basic pay with other components. It processes the payroll and generates the final net salary for each employee

4.3 Allowances & Deductions Module

This module is responsible for calculating additional benefits (allowances) and subtracting deductions such as taxes or penalties to ensure accurate salary computation.

4.4 Database Management Module

This module manages the storage and retrieval of employee data using a database. It ensures that all records are stored securely and can be accessed or modified when needed.

4.5 Reports & Security Module

This module generates payslips and reports for employees and ensures that the system is accessed only by authorized users through a login mechanism.

CHAPTER 5

IMPLEMENTATION

5.1 CODING

```
import tkinter as tk

from tkinter import messagebox, ttk

import csv

import os


FILE_NAME = "employees.csv"


# Create file

if not os.path.exists(FILE_NAME):

    with open(FILE_NAME, "w", newline="") as f:

        writer = csv.writer(f)

        writer.writerow(["Name", "Basic", "Allowance", "Deduction", "Net"])


# ----- LOGIN FUNCTION -----

def login():

    if user_entry.get() == "admin" and pass_entry.get() == "1234":

        login_win.destroy()

        main_app()

    else:

        messagebox.showerror("Error", "Invalid Login")


# ----- MAIN APP -----

def main_app():

    global root, name_e, basic_e, allow_e, deduct_e, tree
```

```

root = tk.Tk()
root.title("Payroll System")
root.geometry("800x550")
root.configure(bg="#0f172a")

tk.Label(root, text="Payroll Processing System",
        font=("Segoe UI", 20, "bold"),
        bg="#0f172a", fg="#38bdf8").pack(pady=10)

frame = tk.Frame(root, bg="#1e293b")
frame.pack(pady=10, padx=20, fill="x")

def label(txt, r):
    tk.Label(frame, text=txt, bg="#1e293b", fg="white",
            font=("Segoe UI", 10)).grid(row=r, column=0, padx=10, pady=5)

def entry(r):
    e = tk.Entry(frame, font=("Segoe UI", 10))
    e.grid(row=r, column=1, pady=5)
    return e

label("Name", 0)
name_e = entry(0)

label("Basic", 1)
basic_e = entry(1)

label("Allowance", 2)

```

```

allow_e = entry(2)

label("Deduction", 3)
deduct_e = entry(3)

# Buttons
btn_frame = tk.Frame(root, bg="#0f172a")
btn_frame.pack(pady=10)

def btn(txt, cmd, col):
    tk.Button(btn_frame, text=txt, command=cmd,
              bg="#38bdf8", fg="black", width=12).grid(row=0, column=col, padx=5)

btn("Add", add_employee, 0)
btn("Update", update_employee, 1)
btn("Delete", delete_employee, 2)
btn("Payslip", generate_payslip, 3)

# TABLE
style = ttk.Style()
style.theme_use("default")
style.configure("Treeview",

                background="#1e293b",
                foreground="white",
                rowheight=25,
                fieldbackground="#1e293b")

tree = ttk.Treeview(root, columns=("Name", "Basic", "Allow", "Deduct", "Net"),

```

```

        show="headings")

for col in ("Name", "Basic", "Allow", "Deduct", "Net"):
    tree.heading(col, text=col)

tree.pack(fill="both", expand=True, padx=20, pady=10)

load_data()

# Click fill form
tree.bind("<ButtonRelease-1>", fill_fields)

root.mainloop()

# ----- LOAD -----
def load_data():
    for i in tree.get_children():
        tree.delete(i)

    with open(FILE_NAME, "r") as f:

reader = csv.reader(f)
    next(reader)
    for row in reader:
        tree.insert("", tk.END, values=row)

# ----- FILL ENTRY -----
def fill_fields(event):
    selected = tree.selection()

```

if selected:

```
data = tree.item(selected[0])["values"]
```

```
name_e.delete(0, tk.END)
```

```
basic_e.delete(0, tk.END)
```

```
allow_e.delete(0, tk.END)
```

```
deduct_e.delete(0, tk.END)
```

```
name_e.insert(0, data[0])
```

```
basic_e.insert(0, data[1])
```

```
allow_e.insert(0, data[2])
```

```
deduct_e.insert(0, data[3])
```

----- ADD -----

```
def add_employee():
```

```
    try:
```

```
        name = name_e.get()
```

```
        basic = float(basic_e.get())
```

```
allow = float(allow_e.get())
```

```
deduct = float(deduct_e.get())
```

```
net = basic + allow - deduct
```

```
with open(FILE_NAME, "a", newline="") as f:
```

```
    writer = csv.writer(f)
```

```
    writer.writerow([name, basic, allow, deduct, net])
```

```
load_data()
```

```

        messagebox.showinfo("Success", "Added")

except:
    messagebox.showerror("Error", "Invalid input")

# ----- DELETE -----

def delete_employee():
    selected = tree.selection()
    if not selected:
        messagebox.showwarning("Select", "Select a row")
        return

    name = tree.item(selected[0])["values"][0]

    rows = []
    with open(FILE_NAME, "r") as f:

        reader = csv.reader(f)
        for row in reader:
            if row and row[0] != name:
                rows.append(row)

    with open(FILE_NAME, "w", newline="") as f:
        csv.writer(f).writerows(rows)

    load_data()
    messagebox.showinfo("Deleted", "Employee removed")

# ----- UPDATE -----

```

```

def update_employee():
    try:
        selected = tree.selection()
        if not selected:
            messagebox.showwarning("Select", "Select a row")
            return

        name = name_e.get()
        basic = float(basic_e.get())
        allow = float(allow_e.get())
        deduct = float(deduct_e.get())

        net = basic + allow - deduct

    rows = []
    with open(FILE_NAME, "r") as f:
        reader = csv.reader(f)
        for row in reader:
            if row[0] == name:
                row = [name, basic, allow, deduct, net]
                rows.append(row)

    with open(FILE_NAME, "w", newline="") as f:
        csv.writer(f).writerows(rows)

    load_data()
    messagebox.showinfo("Updated", "Employee updated")

```

```

except:

    messagebox.showerror("Error", "Invalid input")

# ----- PAYSLIP -----

def generate_payslip():

    selected = tree.selection()

    if not selected:

        return

    data = tree.item(selected[0])["values"]

    messagebox.showinfo("Payslip",

        f"Name: {data[0]}\nBasic: {data[1]}\nAllowance: {data[2]}\nDeduction: {data[3]}\nNet: {data[4]}")

# ----- ATTRACTIVE LOGIN UI -----

login_win = tk.Tk()

login_win.title("Payroll Login")

login_win.geometry("500x400")

login_win.configure(bg="#0f172a")

frame = tk.Frame(login_win, bg="#1e293b", bd=2, relief="ridge")

frame.place(relx=0.5, rely=0.5, anchor="center", width=350, height=300)

tk.Label(frame, text="Payroll System",

    font=("Segoe UI", 18, "bold"),

    bg="#1e293b", fg="#38bdf8").pack(pady=15)

```



```
tk.Label(frame, text="Username",  
         bg="#1e293b", fg="white",  
         font=("Segoe UI", 10)).pack()
```

```
user_entry = tk.Entry(frame, font=("Segoe UI", 11), width=25)  
user_entry.pack(pady=5)
```

```
tk.Label(frame, text="Password",  
         bg="#1e293b", fg="white",  
         font=("Segoe UI", 10)).pack()
```

```
pass_entry = tk.Entry(frame, show="*", font=("Segoe UI", 11), width=25)  
pass_entry.pack(pady=5)
```

```
tk.Button(frame, text="Login", command=login,  
          font=("Segoe UI", 11, "bold"),  
          bg="#38bdf8", fg="black",  
          activebackground="#0ea5e9",  
          width=15).pack(pady=20)
```

```
tk.Label(frame, text="Admin Access Only",  
         bg="#1e293b", fg="gray",  
         font=("Segoe UI", 8)).pack()
```

```
login_win.mainloop()
```

CHAPTER 6

SNAPSHOTS

6.1 Admin Login

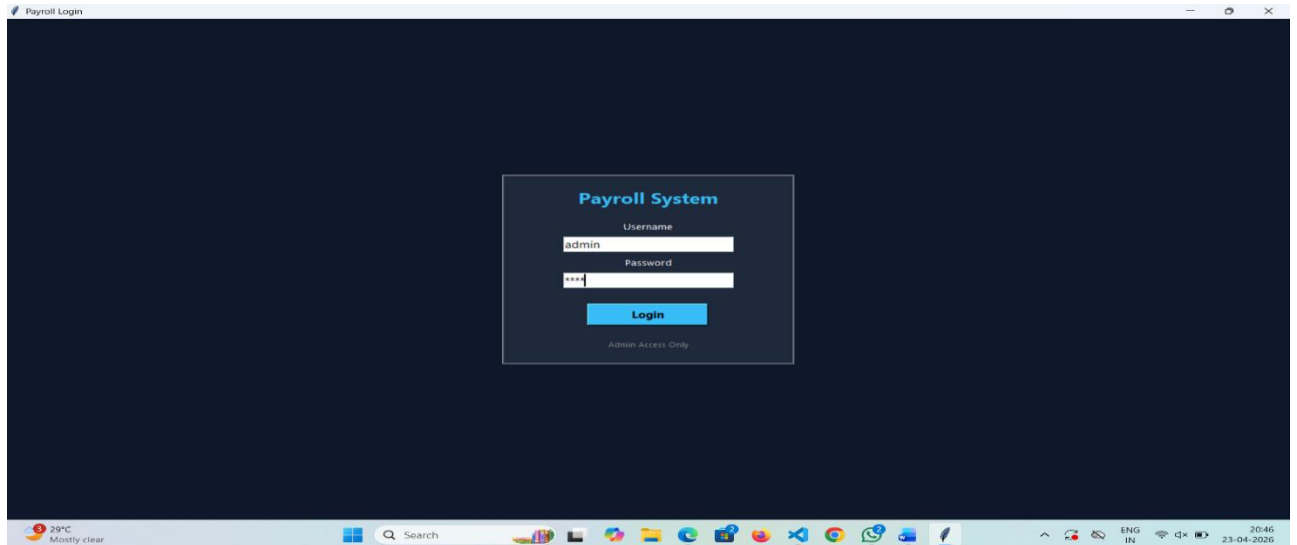


Fig 6.1 Admin Login

6.2 Employee Onboarding – Payroll Entry

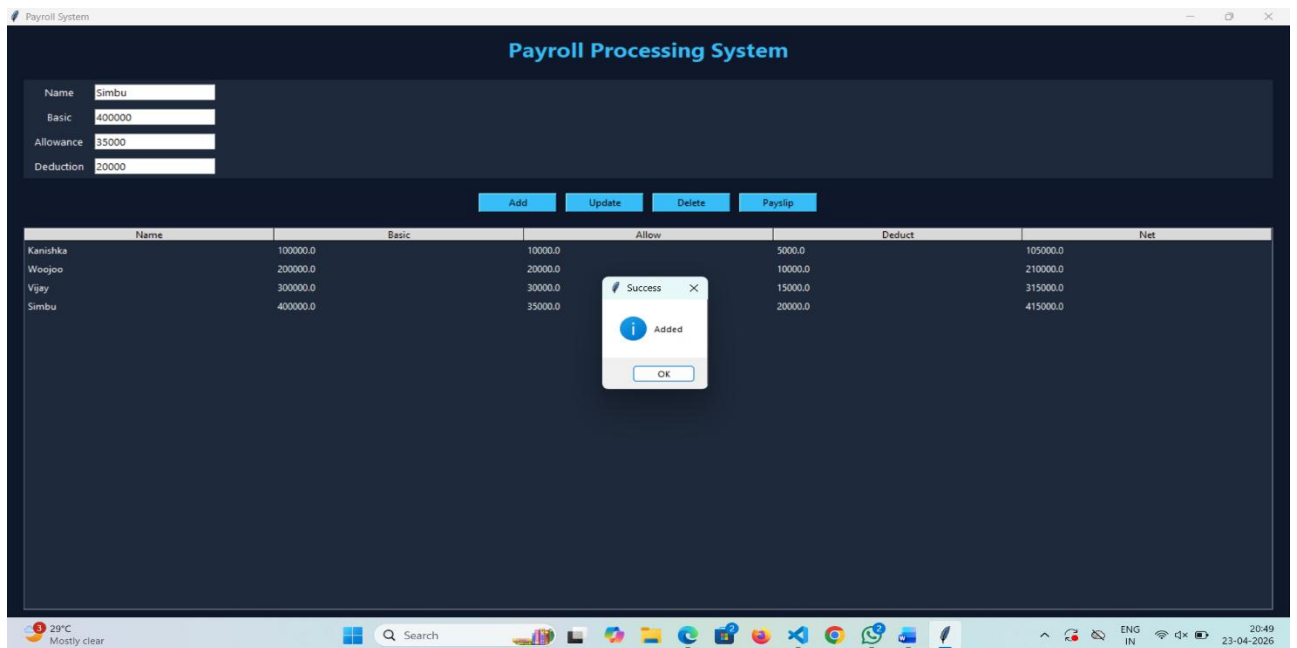


Fig 6.2 Employee Onboarding - Payroll Entry

6.3 Employee Removal Panel

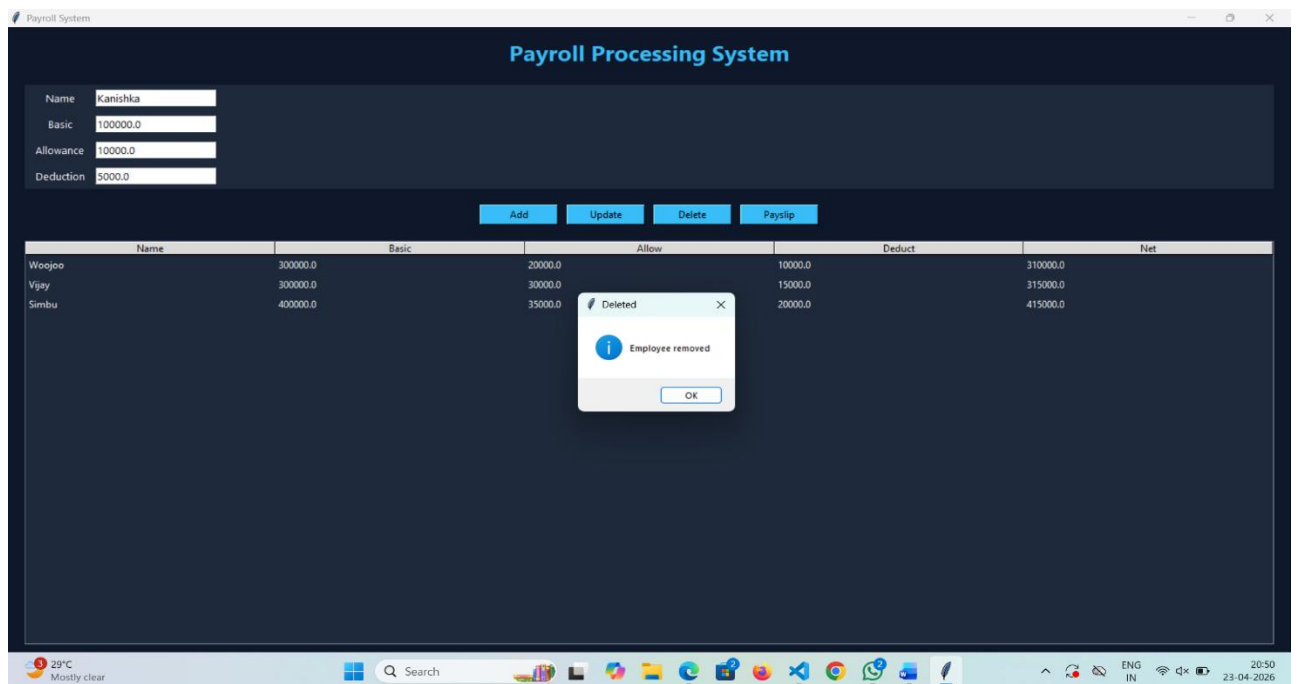


Fig 6.3 Employee Removal Panel

6.4 Employee Payslip Generator

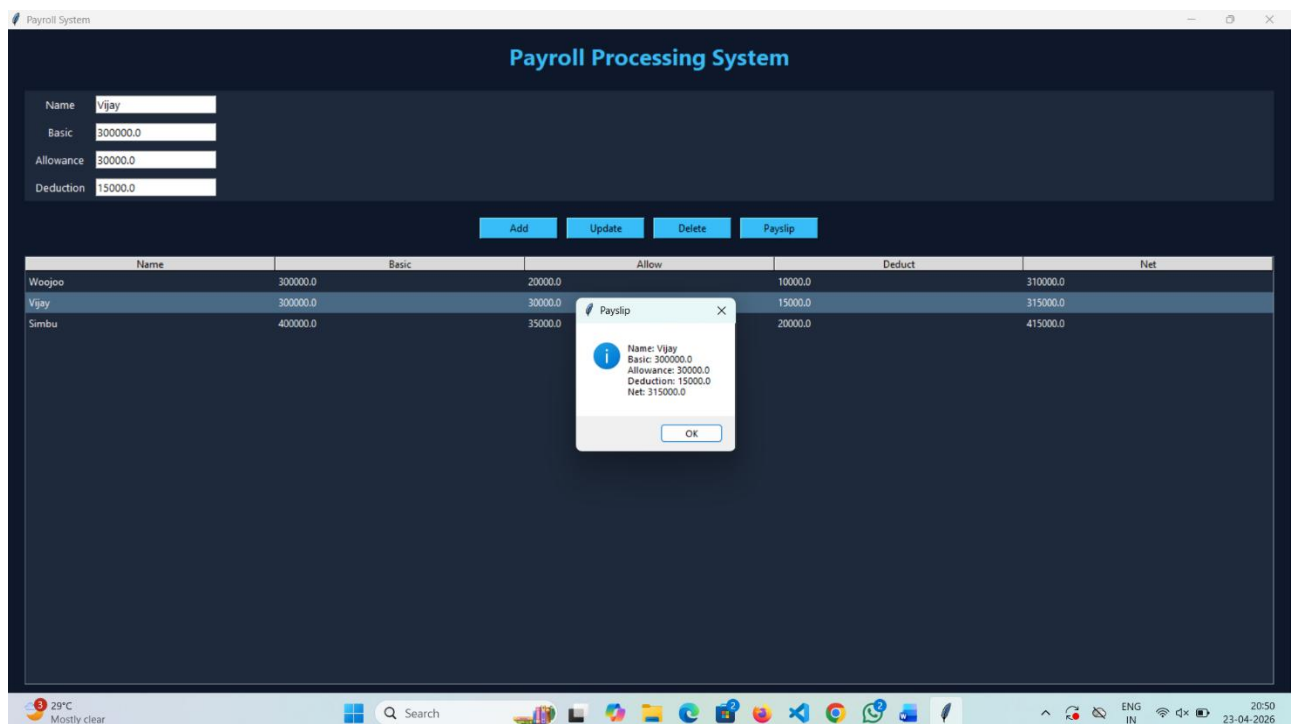


Fig6.4 Employee Payslip Generator

CONCLUSION

The **Payroll Processing System** developed using Python successfully automates the calculation and management of employee salaries. The system reduces manual effort, minimizes errors, and improves efficiency in handling payroll operations. It provides a user-friendly interface for managing employee details, calculating salary components such as allowances and deductions, and generating accurate payslips. The integration of database support ensures secure storage and easy retrieval of employee data. Overall, this project demonstrates the practical application of programming, GUI design, and database concepts to build an effective and reliable payroll management system.

REFERENCES

1. Eric Matthes, Python Crash Course, No Starch Press, 2nd Edition, 2019.
2. Miguel Grinberg, Flask Web Development, O'Reilly Media, 2nd Edition, 2018.
3. Mark Lutz, Learning Python, O'Reilly Media, 5th Edition, 2013.